Networks in R

EM1421 - Social media and web analytics

{Slides courtesy of Prof. Fabiana Zollo}

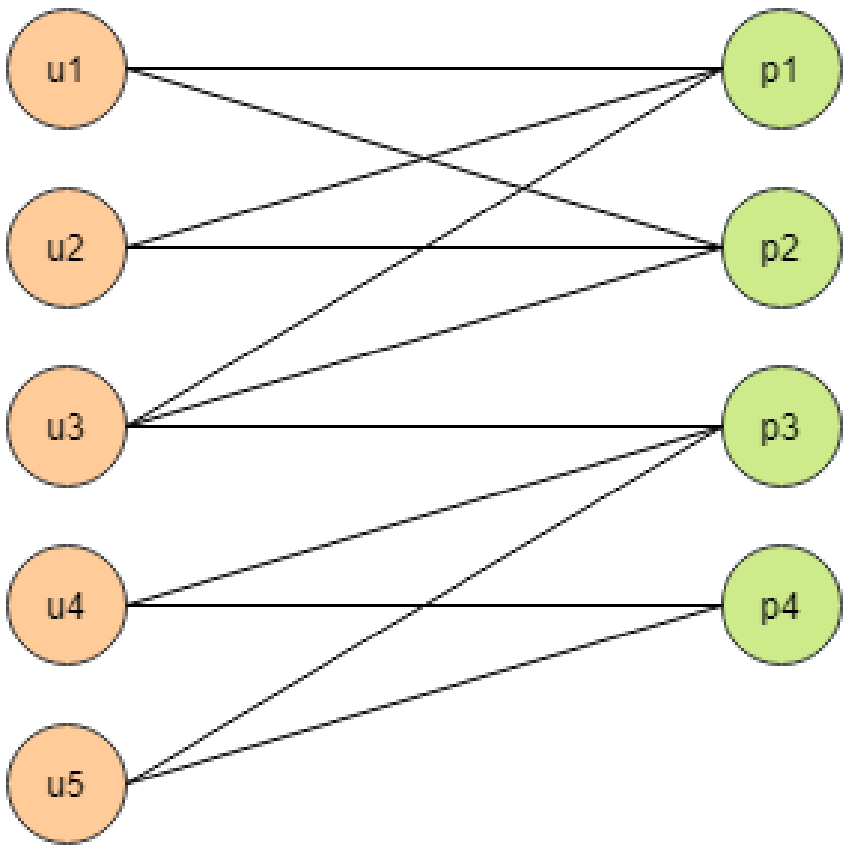
Prof. Pierluigi Zama Ramirez

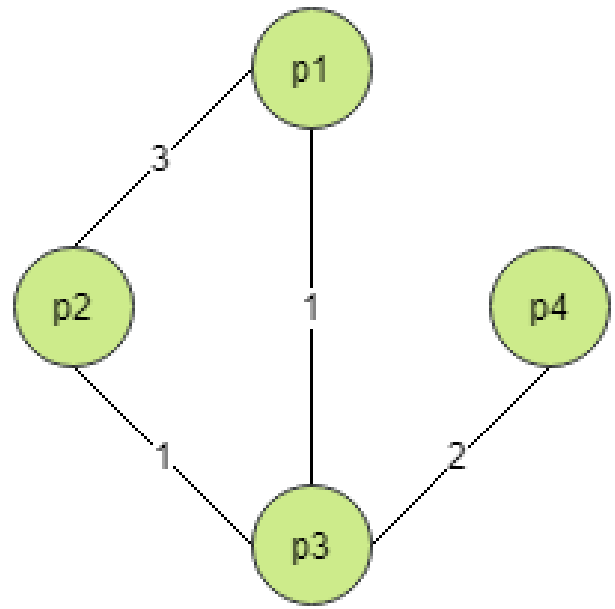
Ca' Foscari University of Venice



Bipartite Networks

- A bipartite graph is a network whose nodes can be divided into two disjoint sets U and V such that each link connects a U-node to a V-node
- If we color the U-nodes green and the V-nodes purple, then each link must connect nodes of different colors

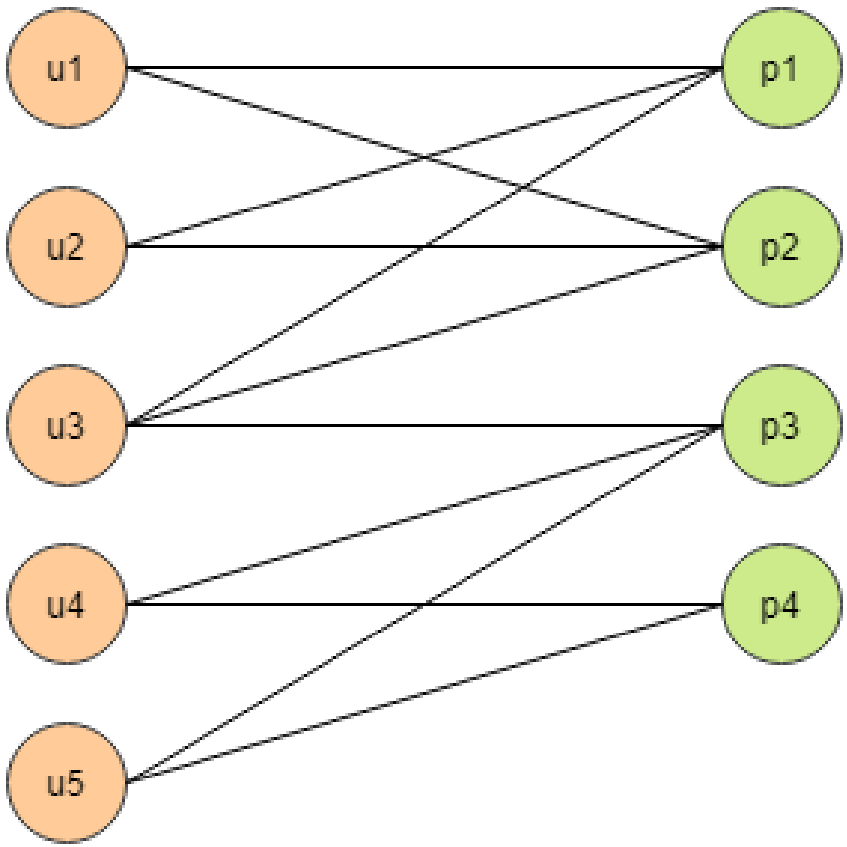


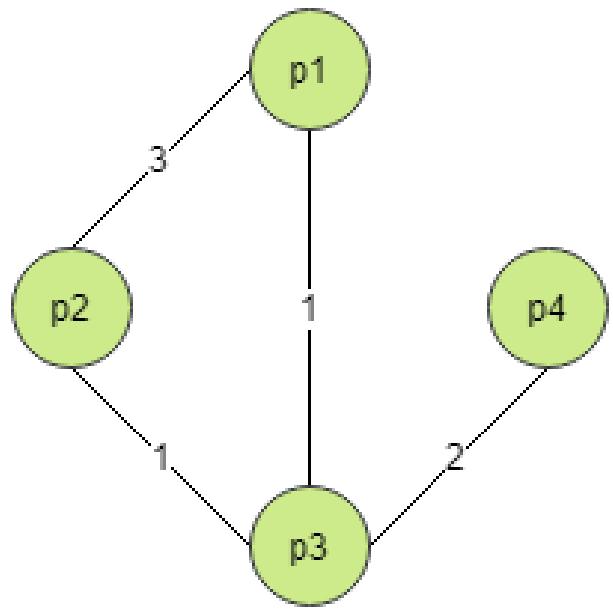


Bipartite Networks

We can generate **two projections** for each bipartite network:

1. The first projection connects two U-nodes by a link if they are linked to the same V-node in the bipartite representation
2. The second projection connects the V-nodes by a link if they connect to the same U-node





Bipartite Networks

Example - The Hollywood actor network

- One set of nodes corresponds to movies (U)
- The other corresponds to actors (V)
- A movie is connected to an actor if the actor plays in that movie
- One projection of this bipartite network is the actor network, in which two nodes are connected to each other if they played in the same movie

Reading network data from files: matrix

```
#Reading data
nodes2 <- read.csv("Dataset2-Media-User-Example-NODES.csv",
links2 <- read.csv("Dataset2-Media-User-Example-EDGES.csv",
```

Reading network data from files

```
# Examine the data:
head(nodes2)
```

```
  id   media media.type media.name audience.size
1 s01     NYT          1  Newspaper            20
2 s02    WaPo          1  Newspaper            25
3 s03     WSJ          1  Newspaper            30
4 s04    USAT          1  Newspaper            32
5 s05 LATimes          1  Newspaper            20
6 s06     CNN          2         TV            56
```

```
head(links2)
```

```
    U01 U02 U03 U04 U05 U06 U07 U08 U09 U10 U11 U12 U13 U14
s01   1   1   1   0   0   0   0   0   0   0   0   0   0   0
s02   0   0   0   1   1   0   0   0   0   0   0   0   0   0
s03   0   0   0   0   0   1   1   1   1   0   0   0   0   0
s04   0   0   0   0   0   0   0   0   1   1   1   0   0   0
s05   0   0   0   0   0   0   0   0   0   0   1   1   1   0
s06   0   0   0   0   0   0   0   0   0   0   0   1   1   1
```

Bipartite: adjacency matrix for a two-mode network

```
1 links2 <- as.matrix(links2)
2 str(links2)
```

```
 int [1:10, 1:20] 1 0 0 0 0 0 0 0 0 1 ...
 - attr(*, "dimnames")=List of 2
  ..$ : chr [1:10] "s01" "s02" "s03" "s04" ...
  ..$ : chr [1:20] "U01" "U02" "U03" "U04" ...
```

```
1 str(nodes2)
```

```
'data.frame':   30 obs. of  5 variables:
 $ id            : chr  "s01" "s02" "s03" "s04" ...
 $ media         : chr  "NYT" "WaPo" "WSJ" "USAT" ...
 $ media.type    : int  1 1 1 1 1 2 2 2 2 2 ...
 $ media.name    : chr  "Newspaper" "Newspaper" "Newspaper"
 $ audience.size: int  20 25 30 32 20 56 34 60 23 34 ...
```

Turning networks into igraph objects

```r
library(igraph)
net2 <- graph_from_incidence_matrix(links2)
net2
```

```
IGRAPH 5167846 UN-B 30 31 --
+ attr: type (v/l), name (v/c)
+ edges from 5167846 (vertex names):
 [1] s01--U01 s01--U02 s01--U03 s02--U04 s02--U05 s02--
U20 s03--U06 s03--U07
 [9] s03--U08 s03--U09 s04--U09 s04--U10 s04--U11 s05--
U11 s05--U12 s05--U13
[17] s06--U13 s06--U14 s06--U17 s07--U14 s07--U15 s07--
U16 s08--U16 s08--U17
[25] s08--U18 s08--U19 s09--U06 s09--U19 s09--U20 s10--
U01 s10--U11
```

Turning networks into igraph objects

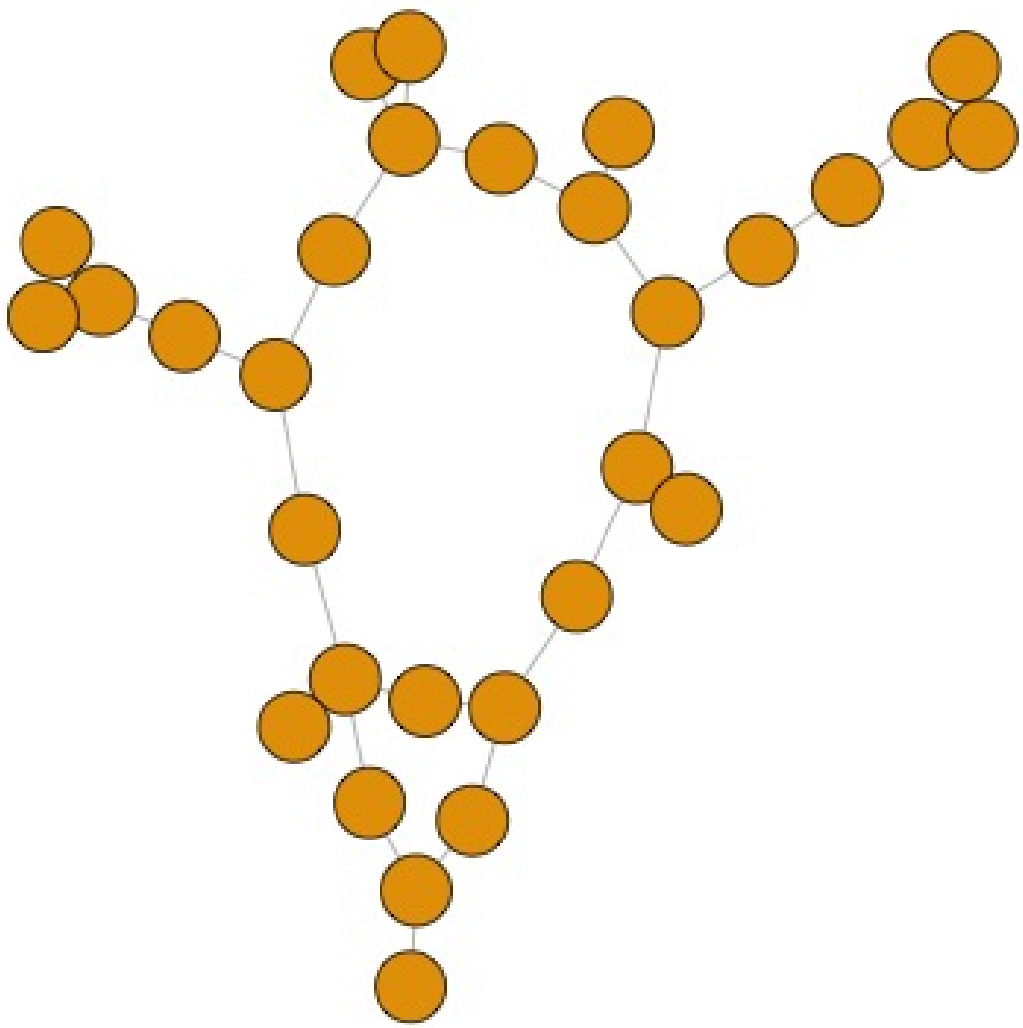
- A built-in vertex attribute 'type' shows which mode vertices belong to

```
table(V(net2)$type)
```

```
FALSE  TRUE
   10    20
```

Bipartite Networks

```
1  plot(net2,vertex.label=NA)
```



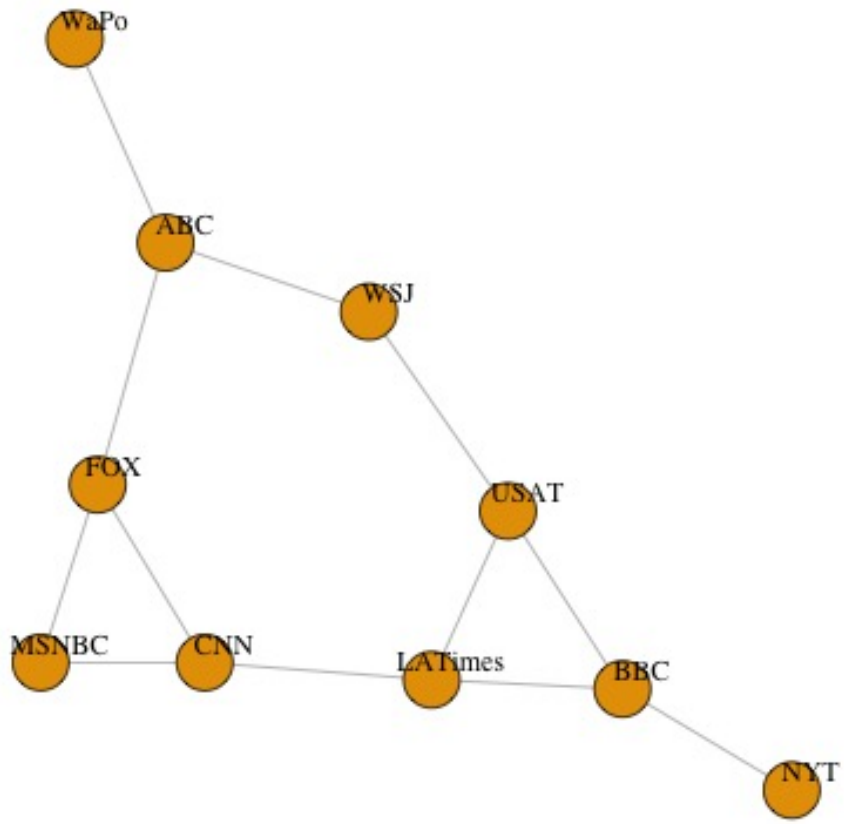
Bipartite Networks

- We can generate bipartite projections for the two-mode network
- Co-memberships are easy to calculate by using `igraph`'s `bipartite.projection` function

```
1 net2.bp <- bipartite.projection(net2)
```

Plotting Bipartite Networks

```
1 plot(net2.bp$proj1, vertex.label.color="black",
2      vertex.label.dist=1,
3      vertex.label=nodes2$media[!is.na(nodes2$media.type)])
```

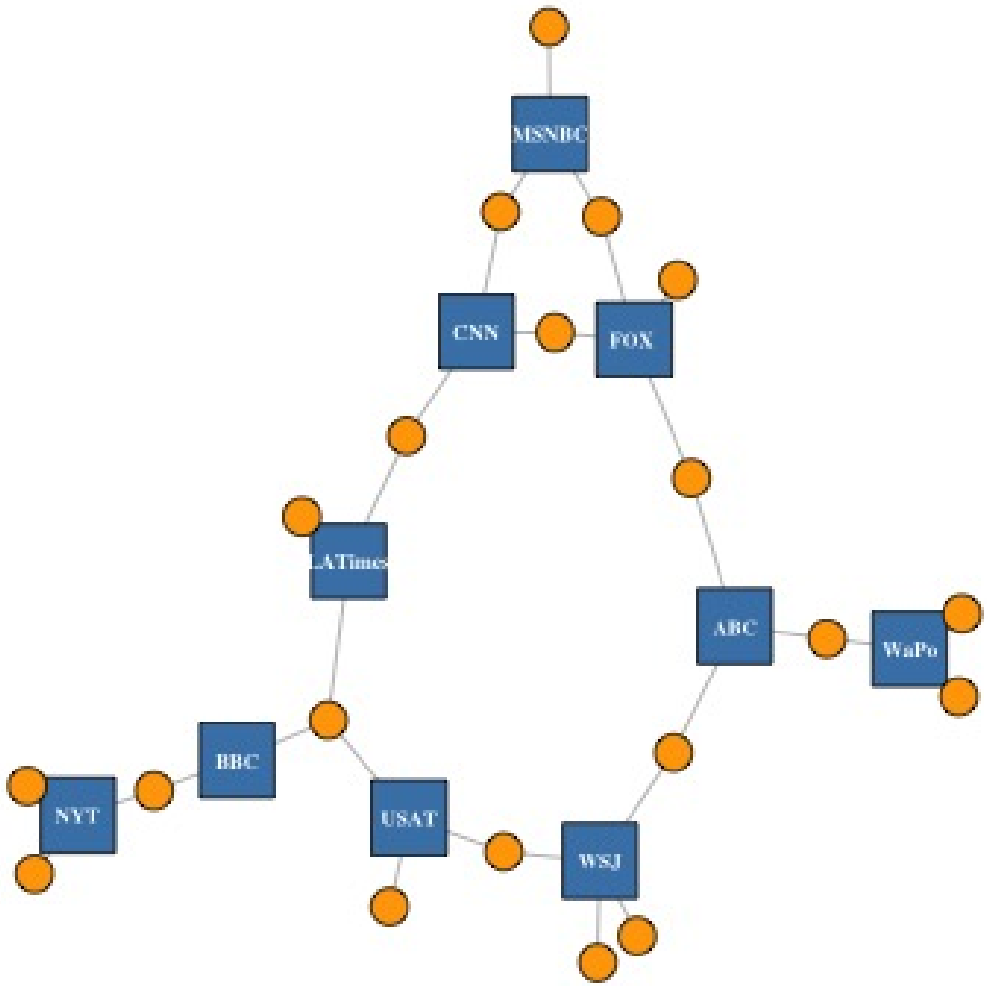


Plotting Bipartite Networks

This time we will make nodes look different based on their type.

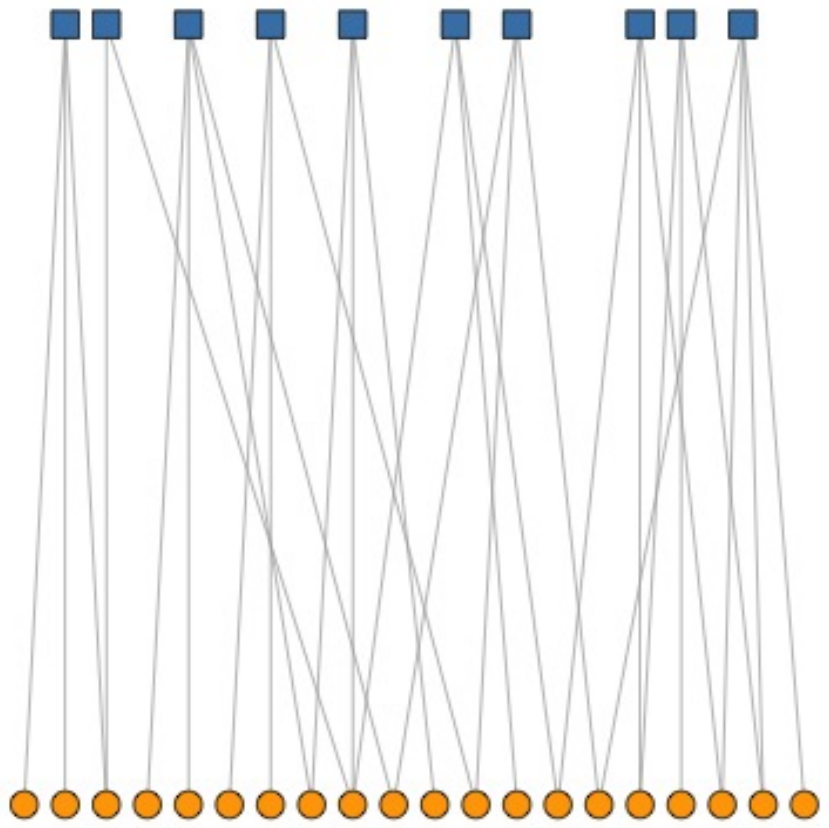
```
V(net2)$color <- c("steel blue", "orange")[V(net2)$type+1]
V(net2)$shape <- c("square", "circle")[V(net2)$type+1]
V(net2)$label <- ""
V(net2)$label[V(net2)$type==F] = nodes2$media[V(net2)$type=
V(net2)$label.cex=.6
V(net2)$label.font=2
plot(net2, vertex.label.color="white",
     vertex.size=(2-V(net2)$type)*8)
```

Plotting Bipartite Networks



Plotting Bipartite Networks

```
1 plot(net2, vertex.label=NA, vertex.size=7,
2      layout=layout_as_bipartite)
```

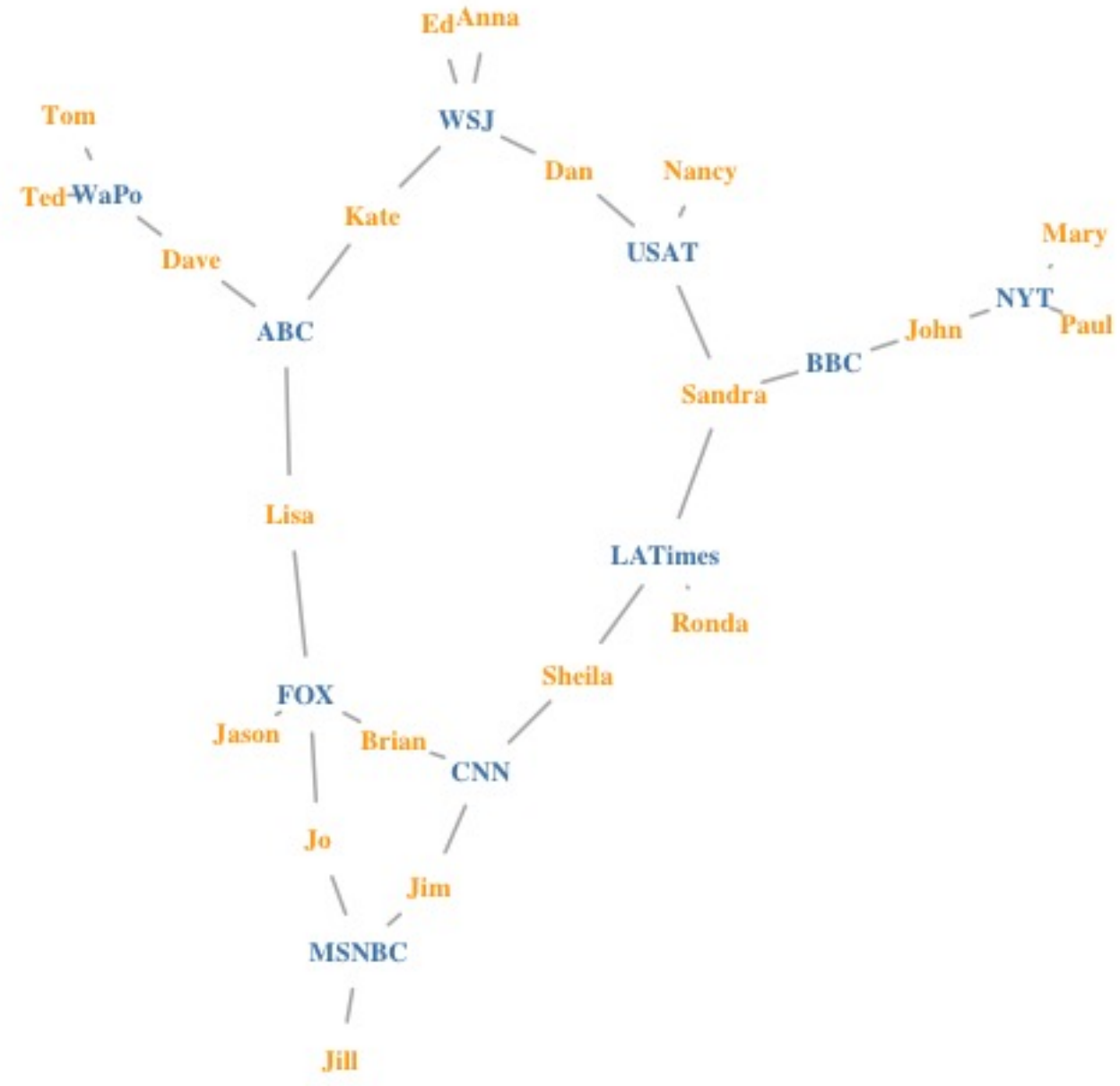


Plotting Bipartite Networks

Using text as nodes:

```
par(mar=c(0,0,0,0))
plot(net2, vertex.shape="none",
     vertex.label=nodes2$media,
     vertex.label.color=V(net2)$color,
     vertex.label.font=2,
     vertex.label.cex=.95, edge.color="gray70",
     edge.width=2)
```

Plotting Bipartite Networks

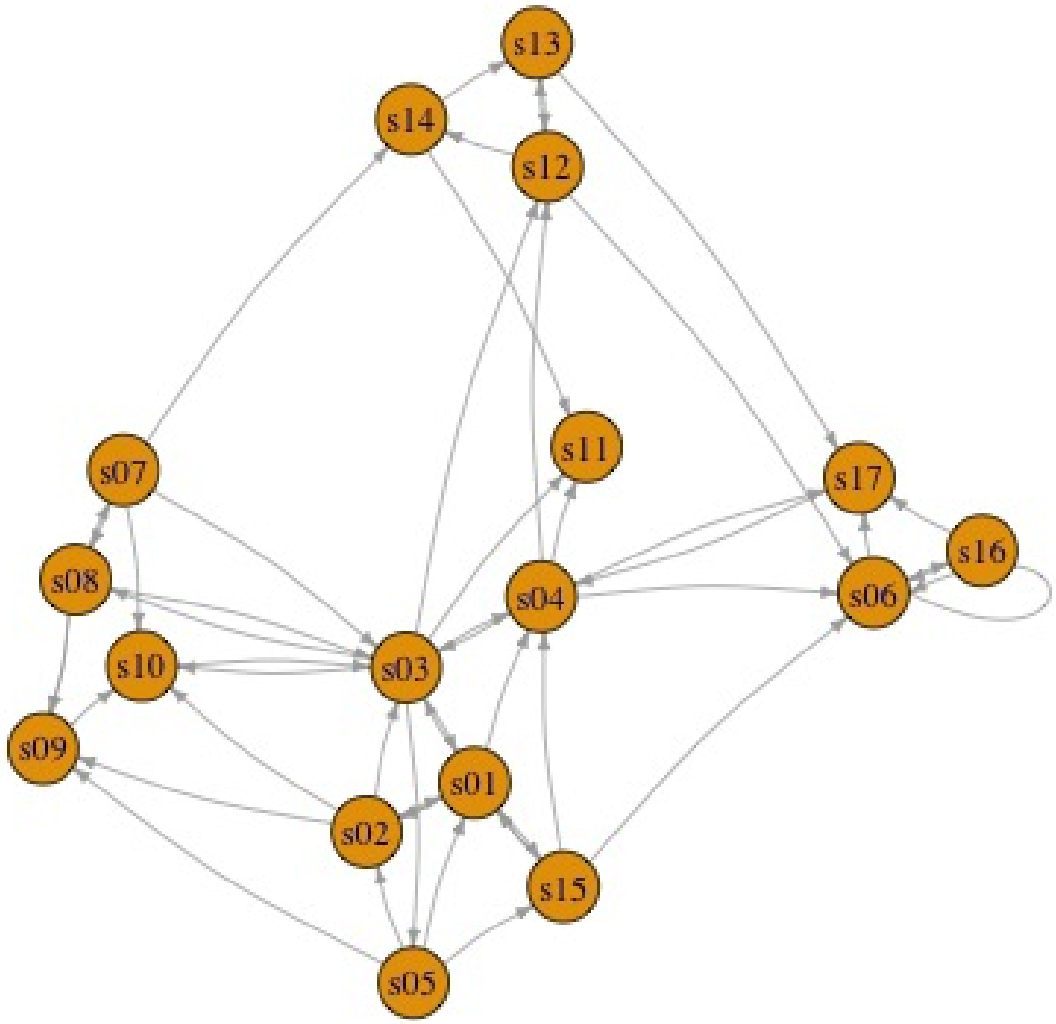


More on Graph Visualization

- We can set the node & edge options in two ways - one is to specify them in the plot() function, as we are doing below.
- Plot with curved edges (edge.curved=.1) and reduce arrow size:

```
library(igraph)
nodes <- read.csv("Dataset1-Media-Example-NODES.csv",
                  header=T, as.is=T)
links <- read.csv("Dataset1-Media-Example-EDGES.csv",
                  header=T, as.is=T)
net <- graph_from_data_frame(d=links, vertices=nodes,
                             directed=T)
plot(net, edge.arrow.size=.4, edge.curved=.1)
```

Plotting in igraph

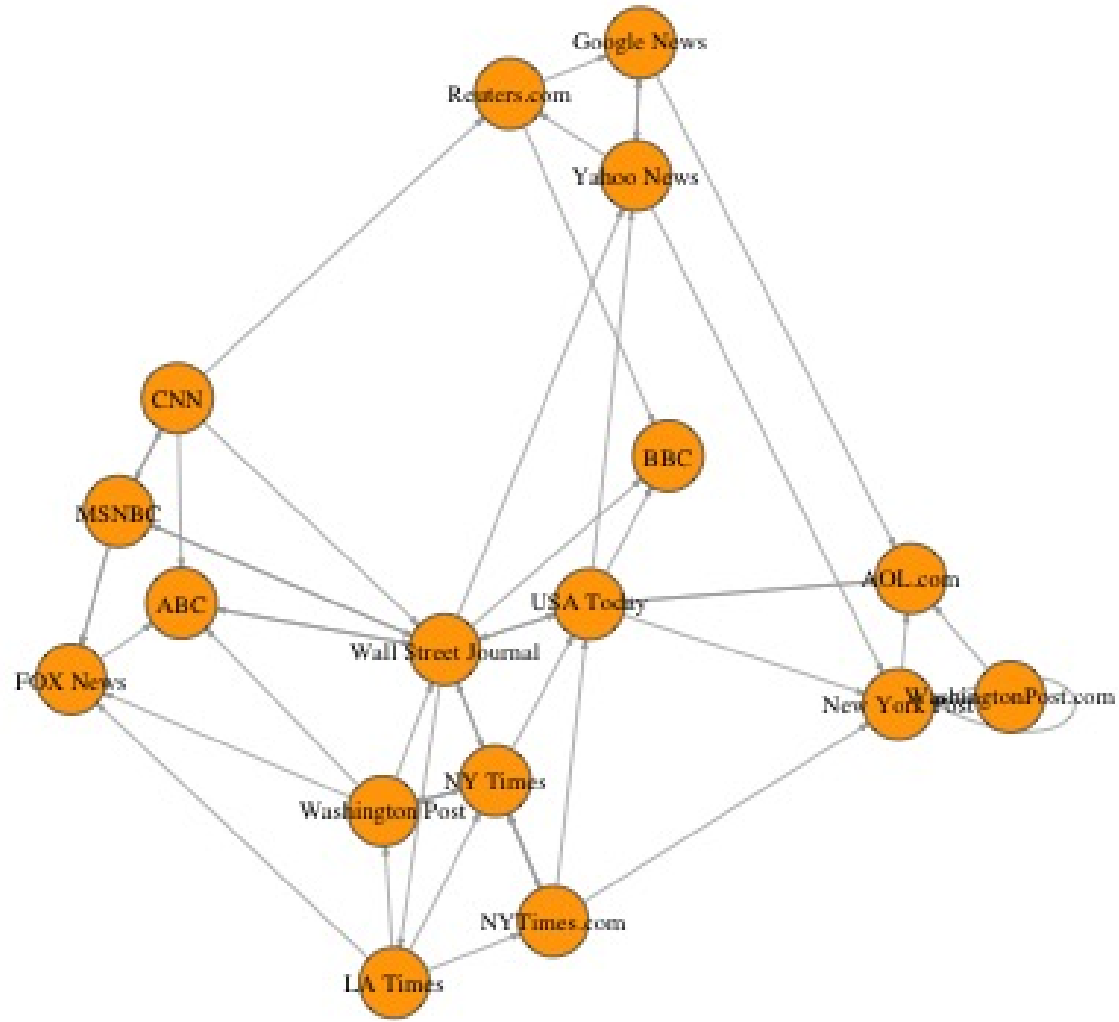


Plotting in igraph

- Set node color to orange and the border color to hex #555555
- Replace the vertex label with the node names stored in "media"

```
plot(net, edge.arrow.size=.2, edge.curved=0,
     vertex.color="orange", vertex.frame.color="#555555",
     vertex.label=V(net)$media, vertex.label.color="black",
     vertex.label.cex=.7)
```

Plotting in igraph

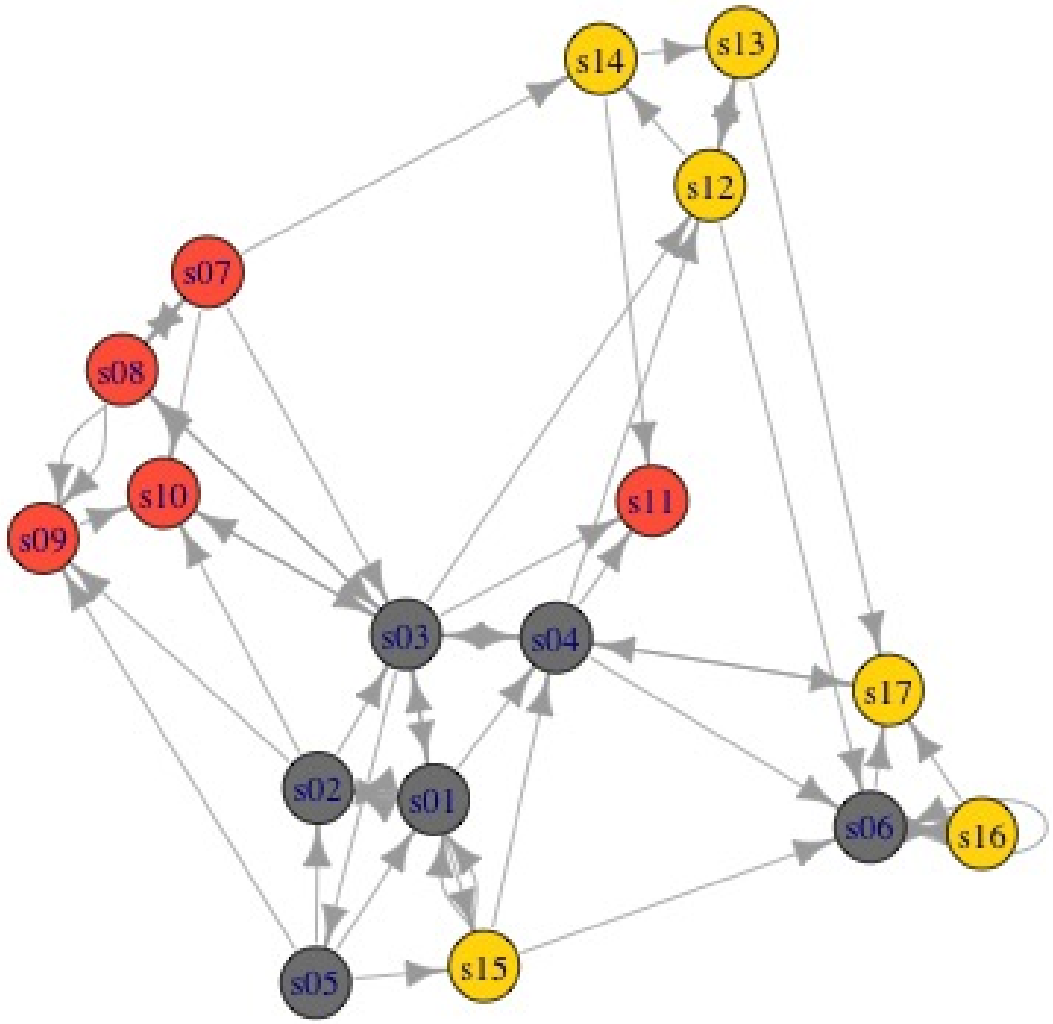


Plotting in igraph

The second way to set attributes is to add them to the igraph object. - Generate colors based on media type:

```
colrs <- c("gray50", "tomato", "gold")
V(net)$color <- colrs[V(net)$media.type]
plot(net)
```

Plotting in igraph

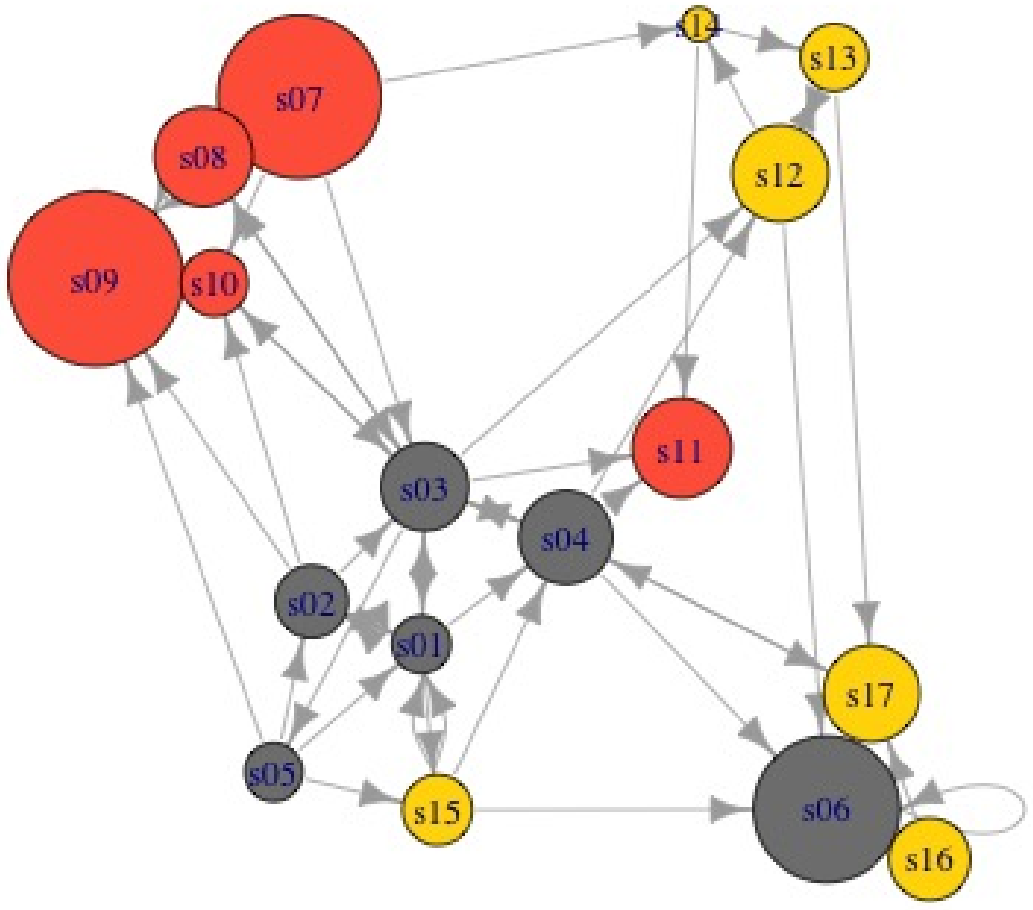


Plotting in igraph

- Set node size based on audience size:

```
1 V(net)$size <- V(net)$audience.size*0.7
2 plot(net)
```

Plotting in igraph

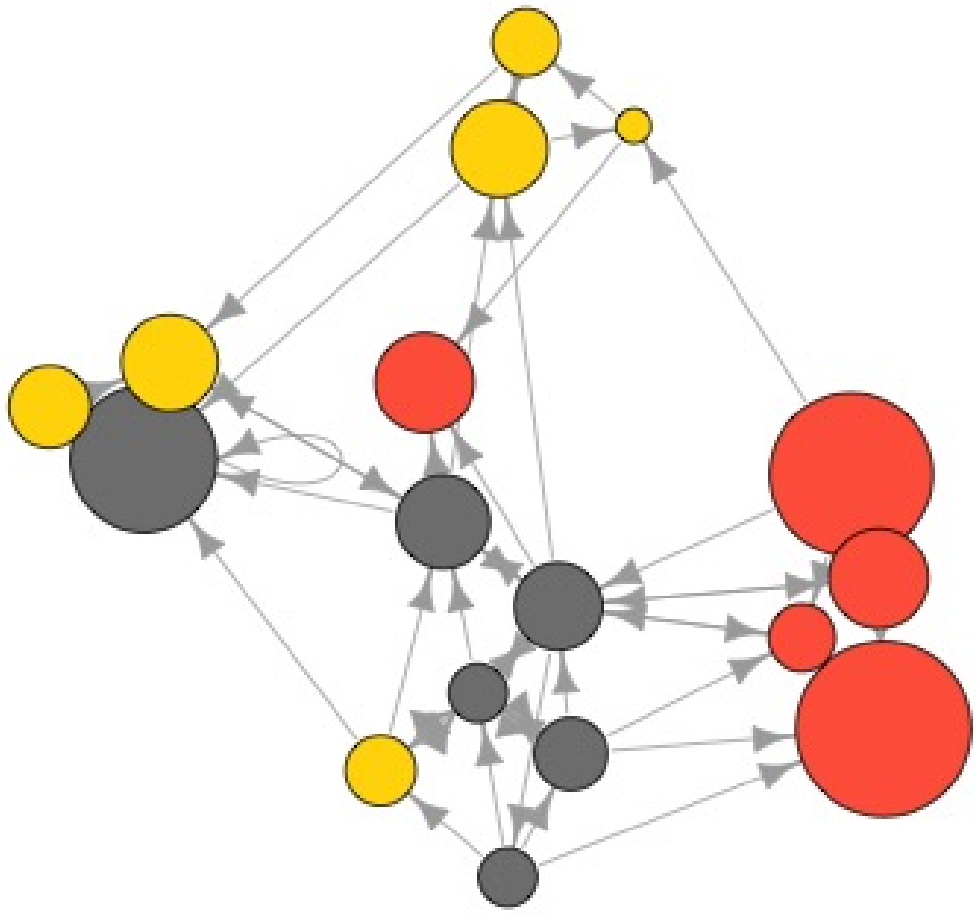


Plotting in igraph

- The labels are currently node IDs
- Setting them to NA will render no labels:

```
1 V(net)$label.color <- "black"
2 V(net)$label <- NA
3 plot(net)
```

Plotting in igraph

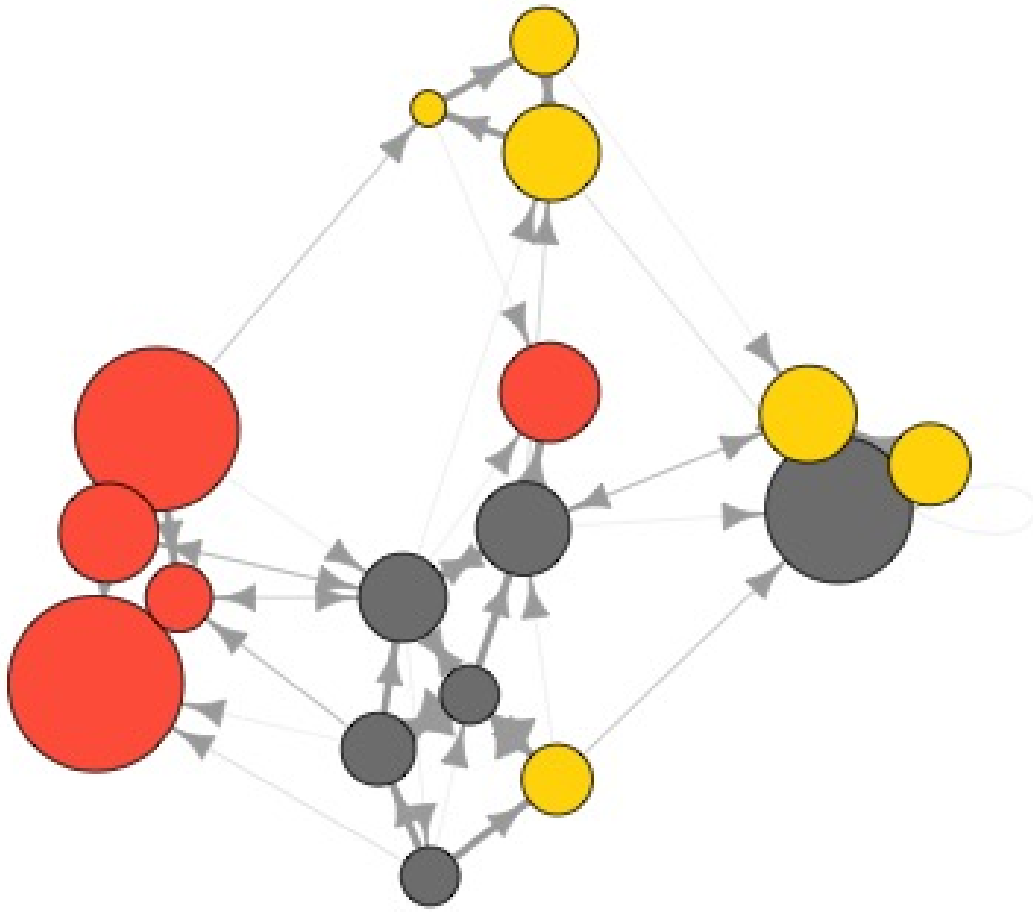


Plotting in igraph

- Set edge width based on weight:

```
1 E(net)$width <- E(net)$weight/6
2 plot(net)
```

Plotting in igraph

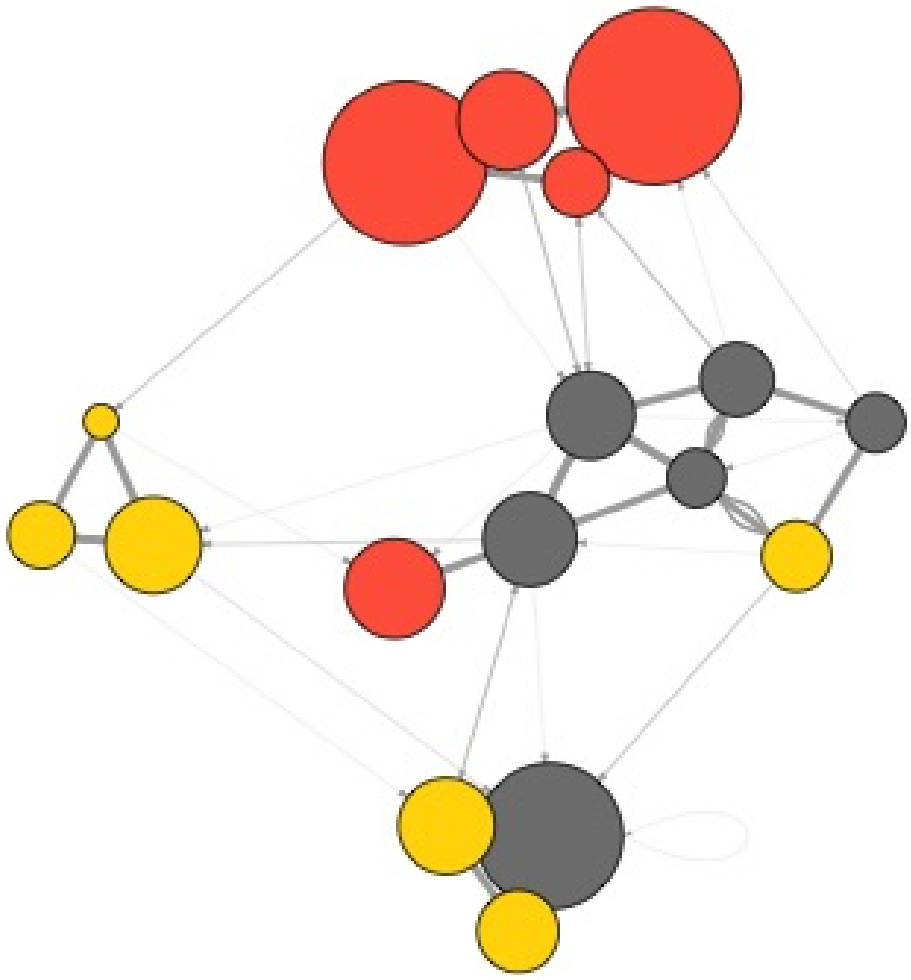


Plotting in igraph

- Change arrow size and edge color:

```
E(net)$arrow.size <- .2
E(net)$edge.color <- "gray80"
plot(net)
```

Plotting in igraph

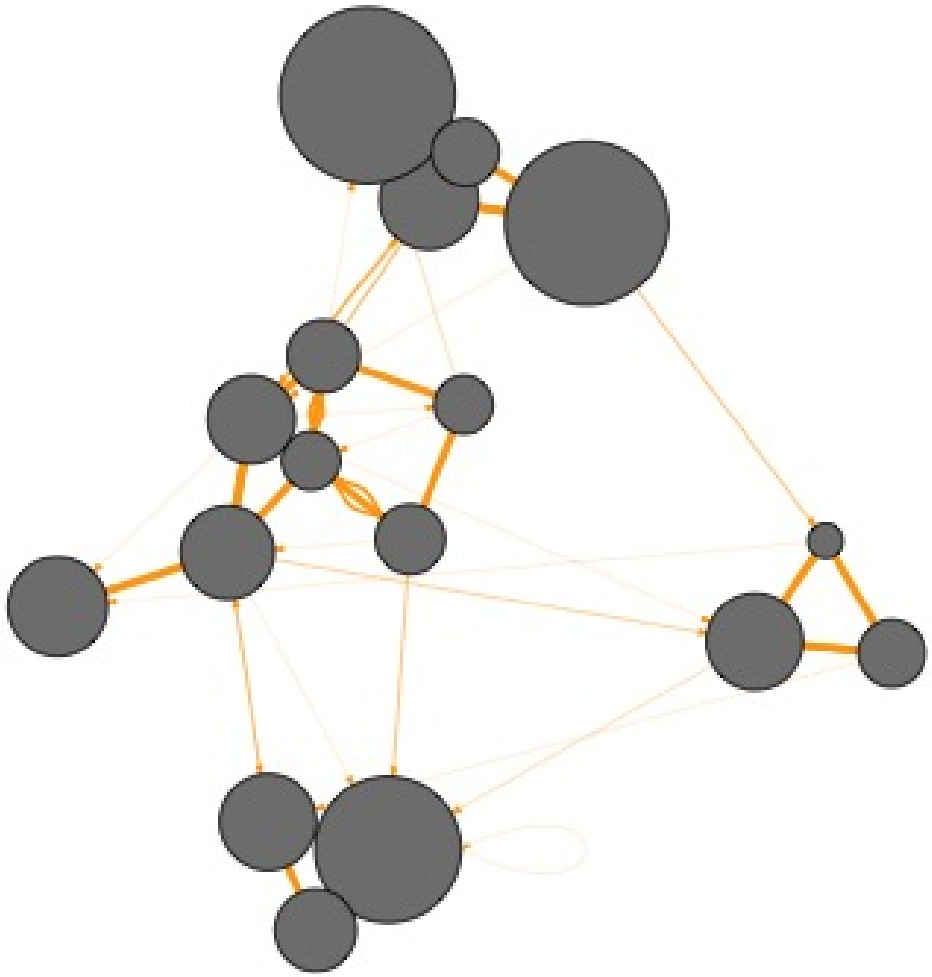


Plotting in igraph

- We can override the attributes explicitly in the plot:

```
1 plot(net, edge.color="orange", vertex.color="gray50")
```

Plotting in igraph

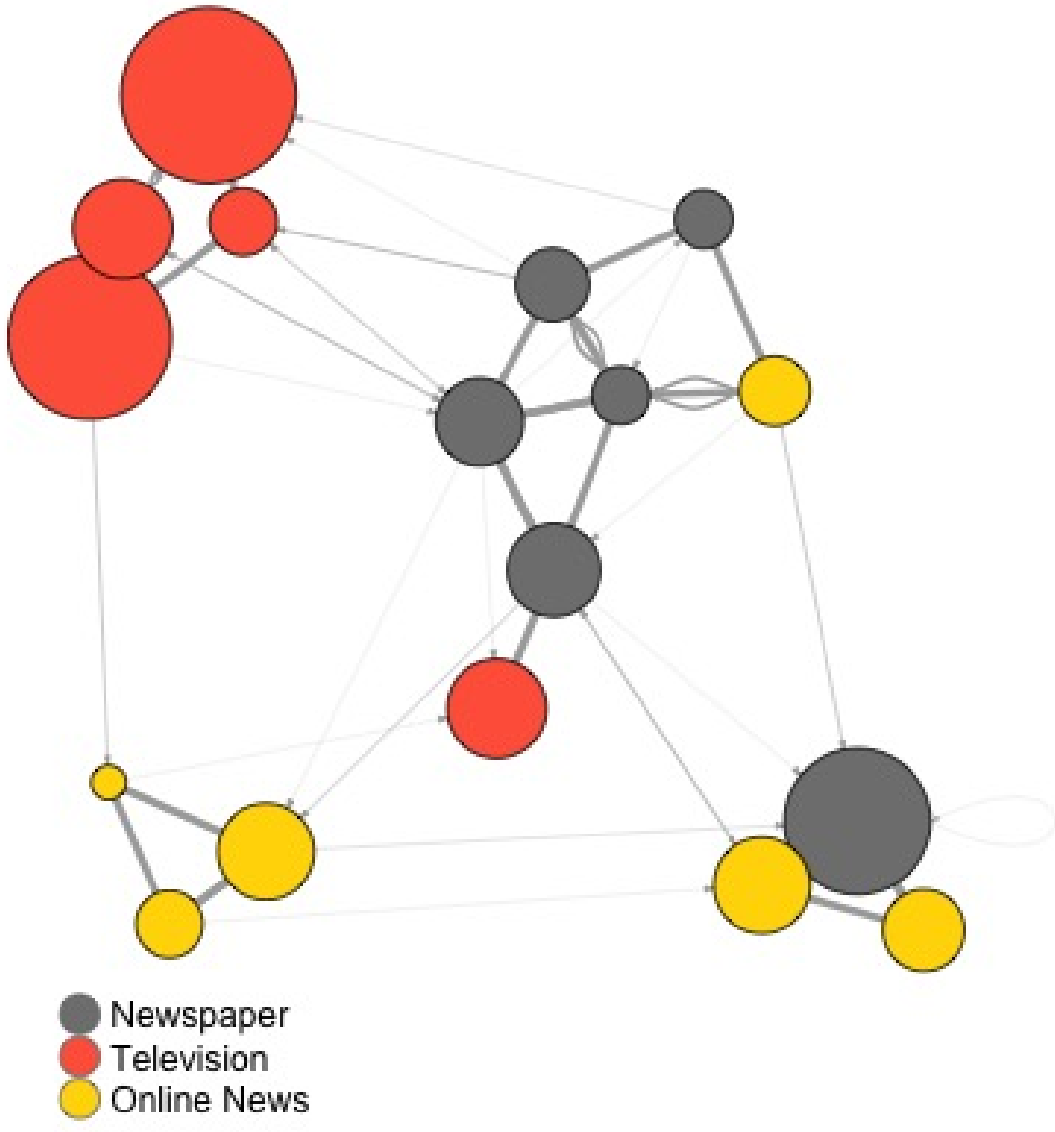


Plotting in igraph

- We can add a legend:

```
plot(net)
legend(x=-1.1, y=-1.1,
       c("Newspaper","Television", "Online News"),
       pch=21, col="#777777", pt.bg=colrs, pt.cex=2.5,
       bty="n", ncol=1)
```

Plotting in igraph

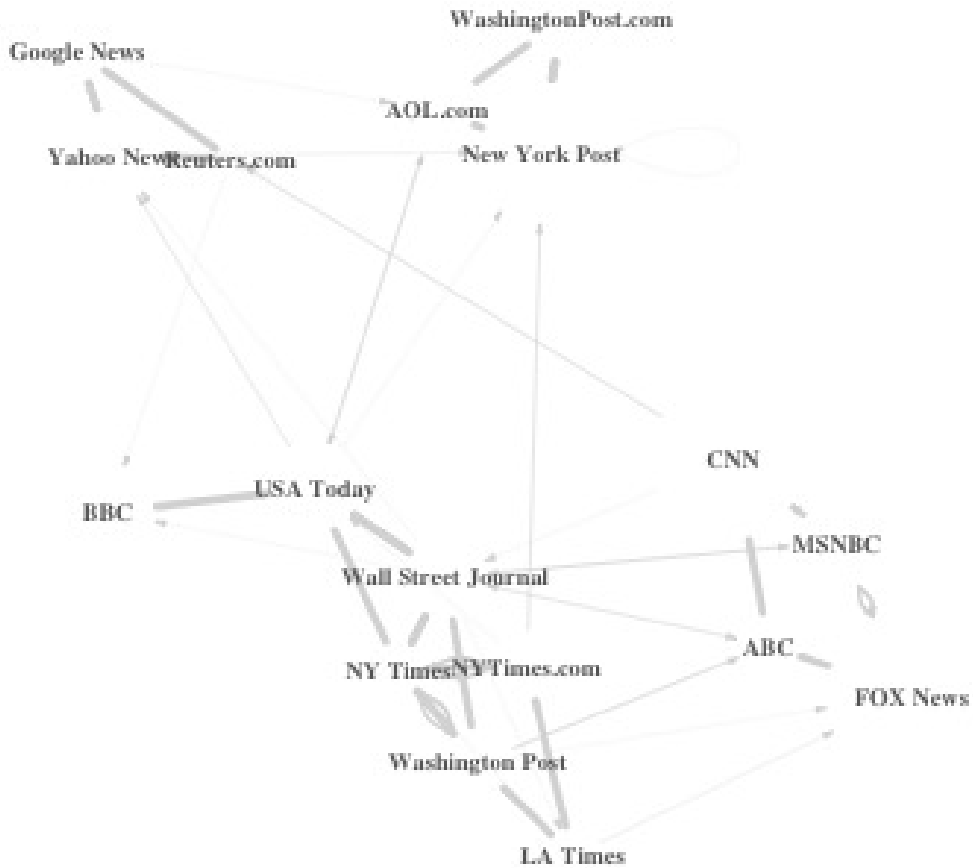


Plotting in igraph

- Sometimes, especially with semantic networks, we may be interested in plotting only the labels of the nodes:

```
plot(net, vertex.shape="none", vertex.label=V(net)$media,
     vertex.label.font=2, vertex.label.color="gray40",
     vertex.label.cex=.7, edge.color="gray85")
```

Plotting in igraph

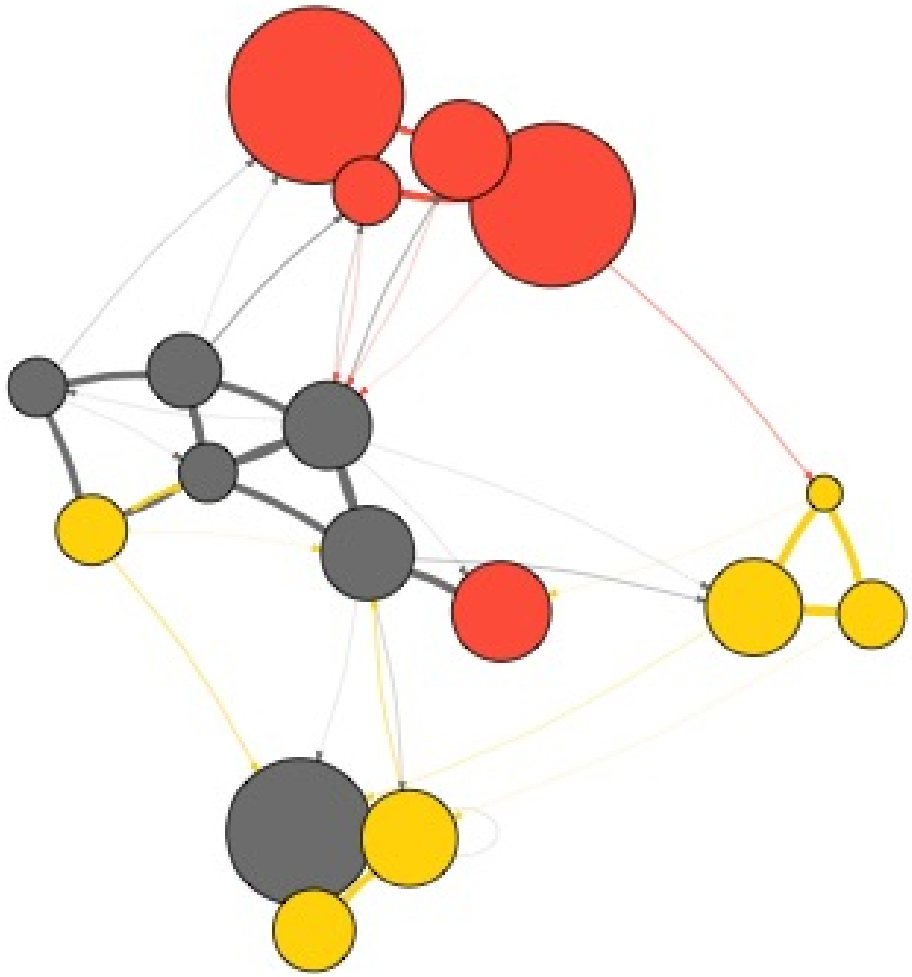


Plotting in igraph

- Let's color the edges of the graph based on the color of their source node

```
edge.start <- ends(net, es=E(net), names=F)[,1]
#get the starting node for each edge
edge.col <- V(net)$color[edge.start]
my_layout = layout.fruchterman.reingold(net)
plot(net, edge.color=edge.col,
     edge.curved=.1, layout = my_layout)
```

Plotting in igraph



Layouts

Let's generate a larger graph:

```
net.bg <- sample_pa(80) #BA-model is a very simple stochast
V(net.bg)$size <- 8
V(net.bg)$frame.color <- "white"
V(net.bg)$color <- "orange"
V(net.bg)$label <- ""
E(net.bg)$arrow.mode <- 0
plot(net.bg)
```

Layouts

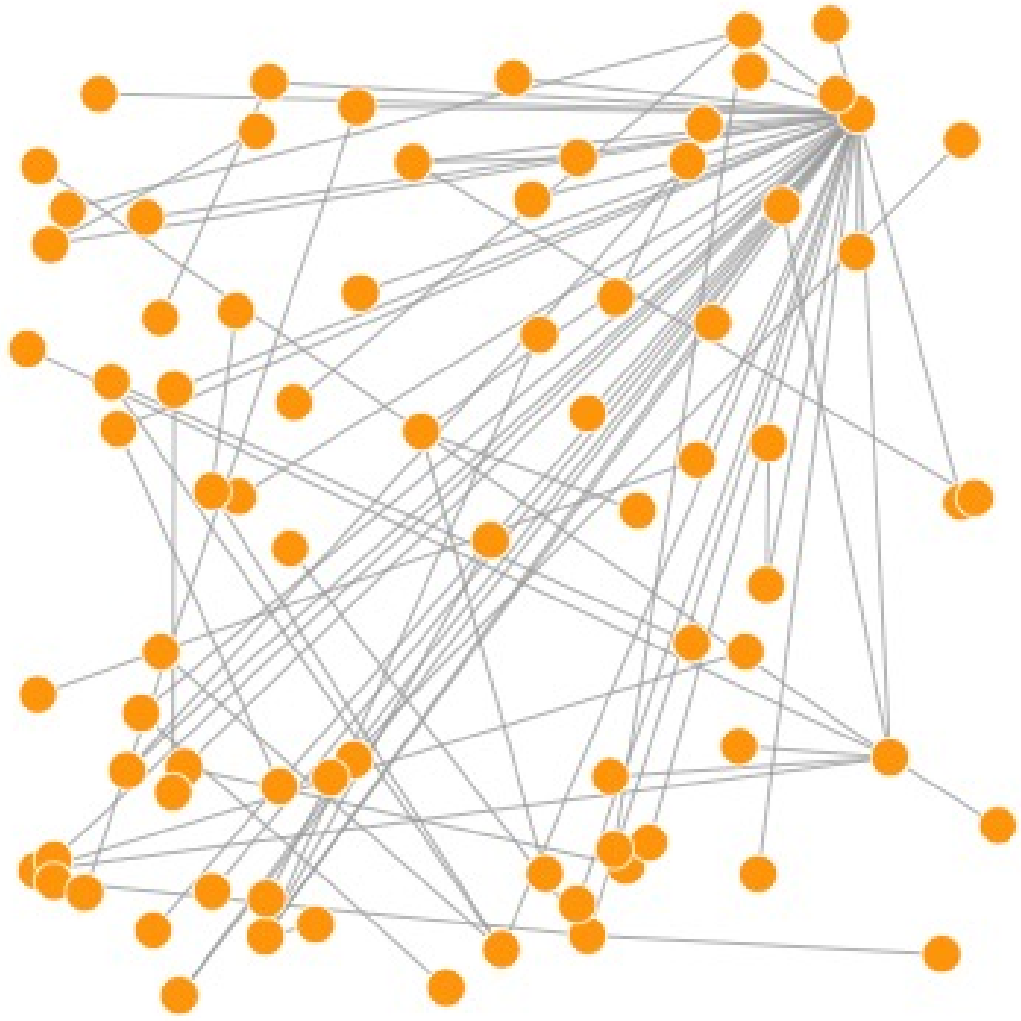


Layouts

- Network layouts are algorithms that return coordinates for each node in a network
- You can set the layout in the plot function:

```
1 plot(net.bg, layout=layout_randomly)
```

Layouts

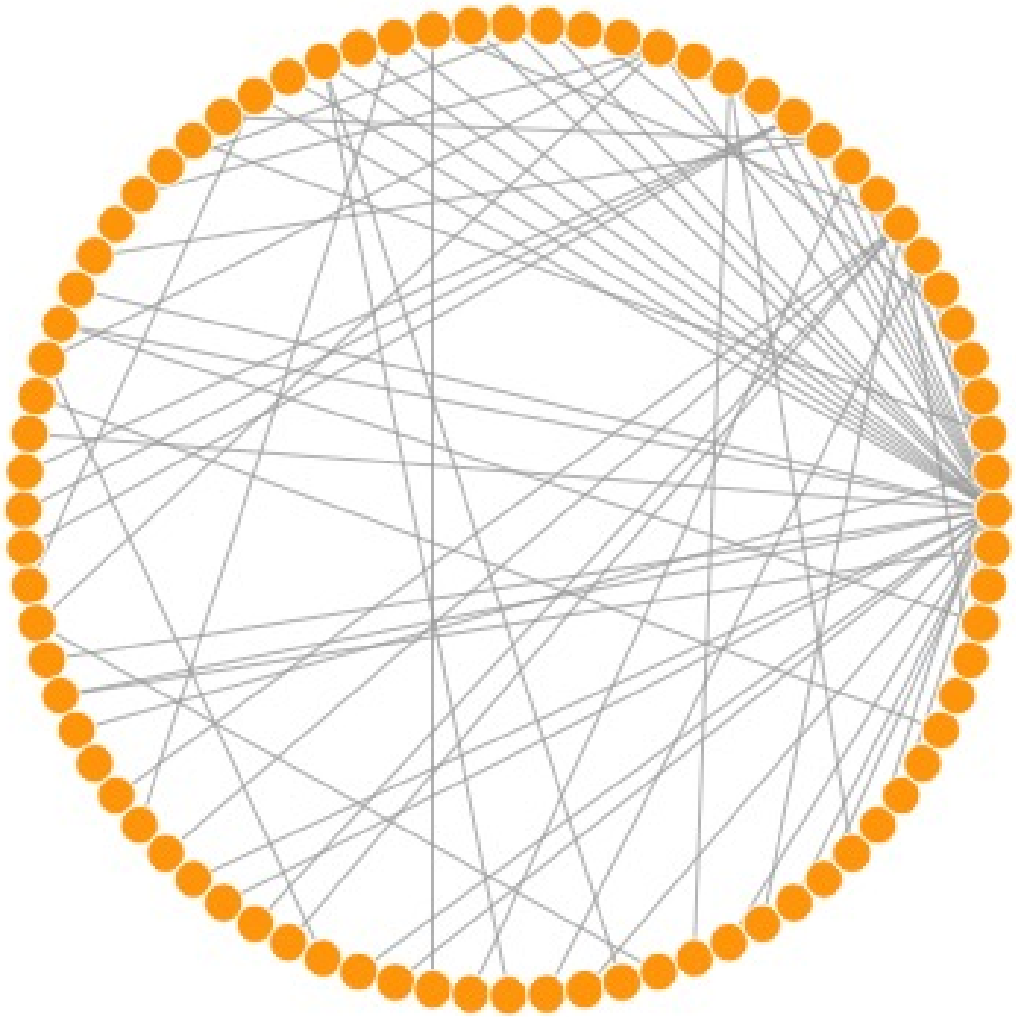


Layouts

Or calculate the vertex coordinates in advance:

```
1 l <- layout_in_circle(net.bg)
2 plot(net.bg, layout=l)
```

Layouts

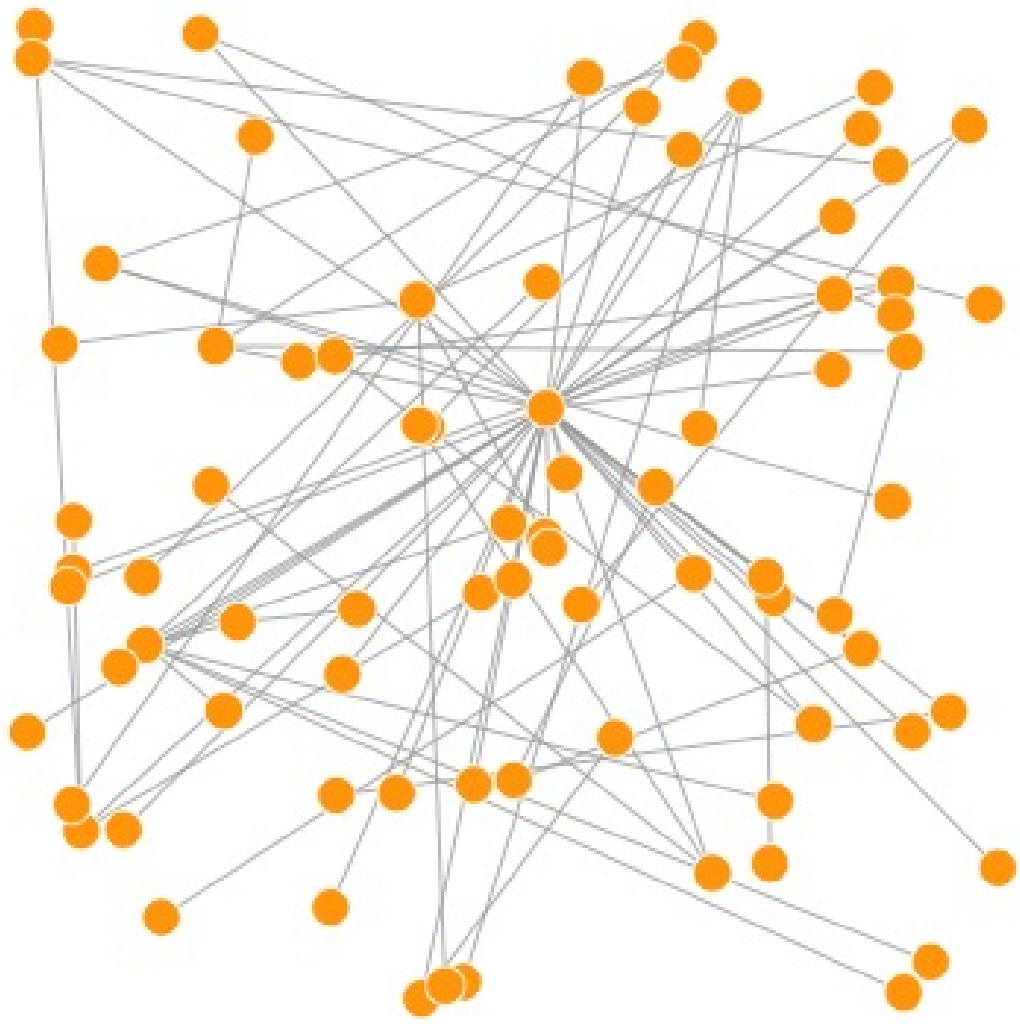


Layouts

Randomly placed vertices:

```
1 l <- layout_randomly(net.bg)
2 plot(net.bg, layout=l)
```

Layouts

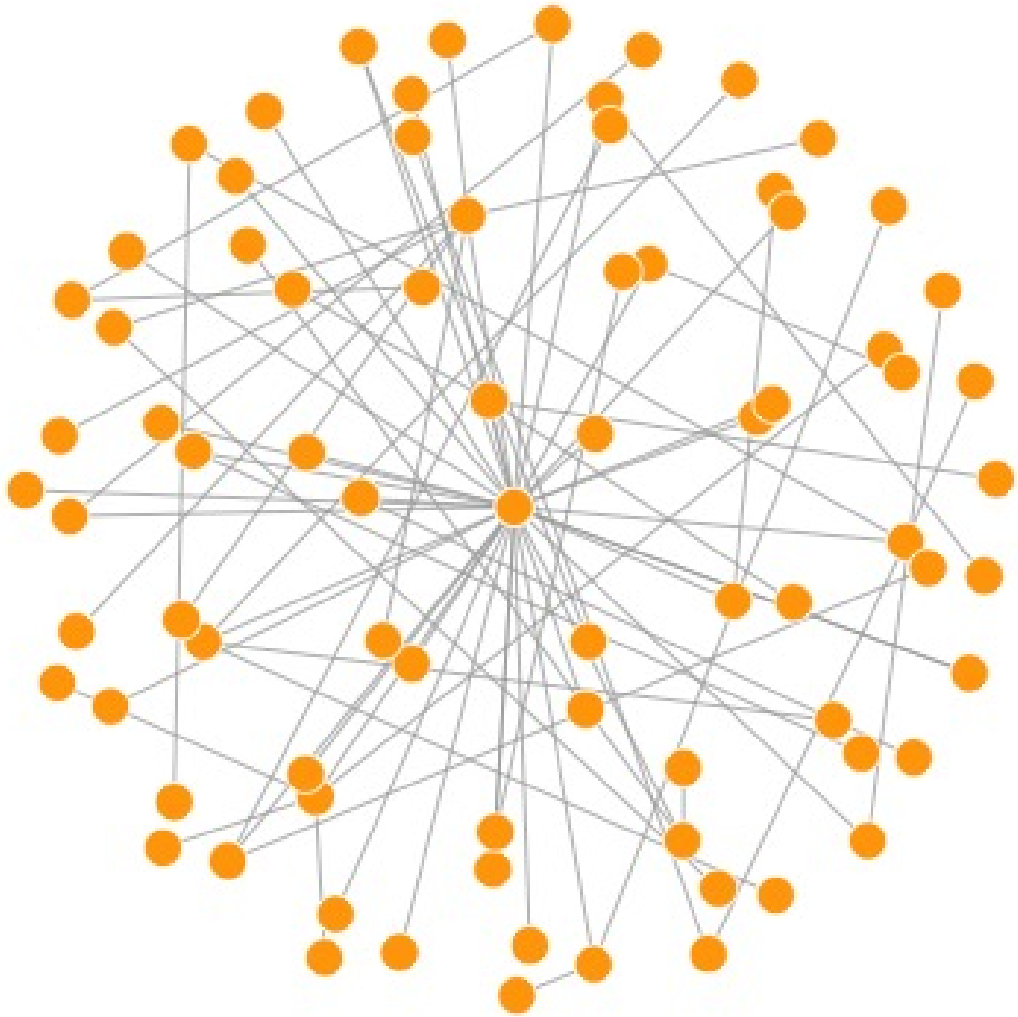


Layouts

- 3D sphere layout

```
1 l <- layout_on_sphere(net.bg)
2 plot(net.bg, layout=l)
```

Layouts

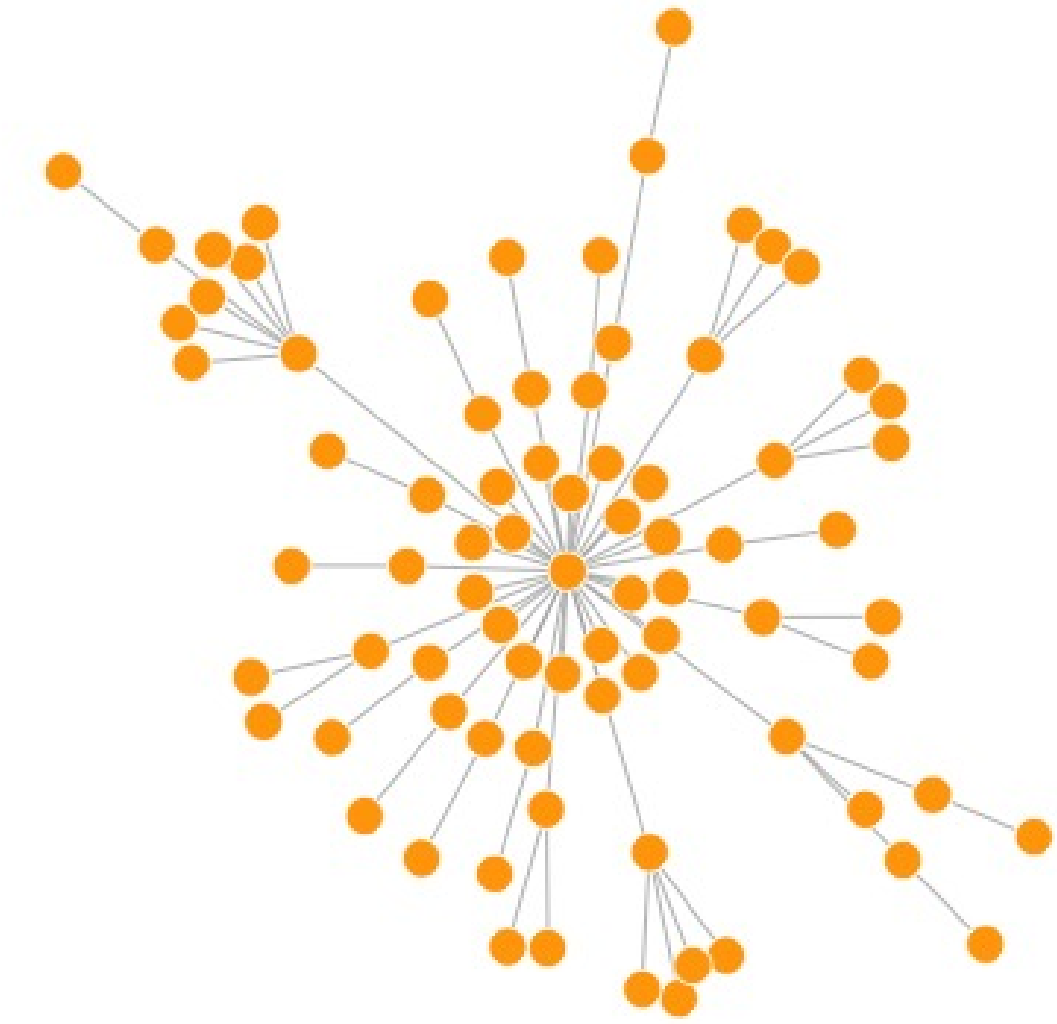


Layouts

- The Fruchterman-Reingold force-directed algorithm
- Nice but slow (recommended up to ~1,000 vertices)

```
l <- layout_with_fr(net.bg)
plot(net.bg, layout=l)
```

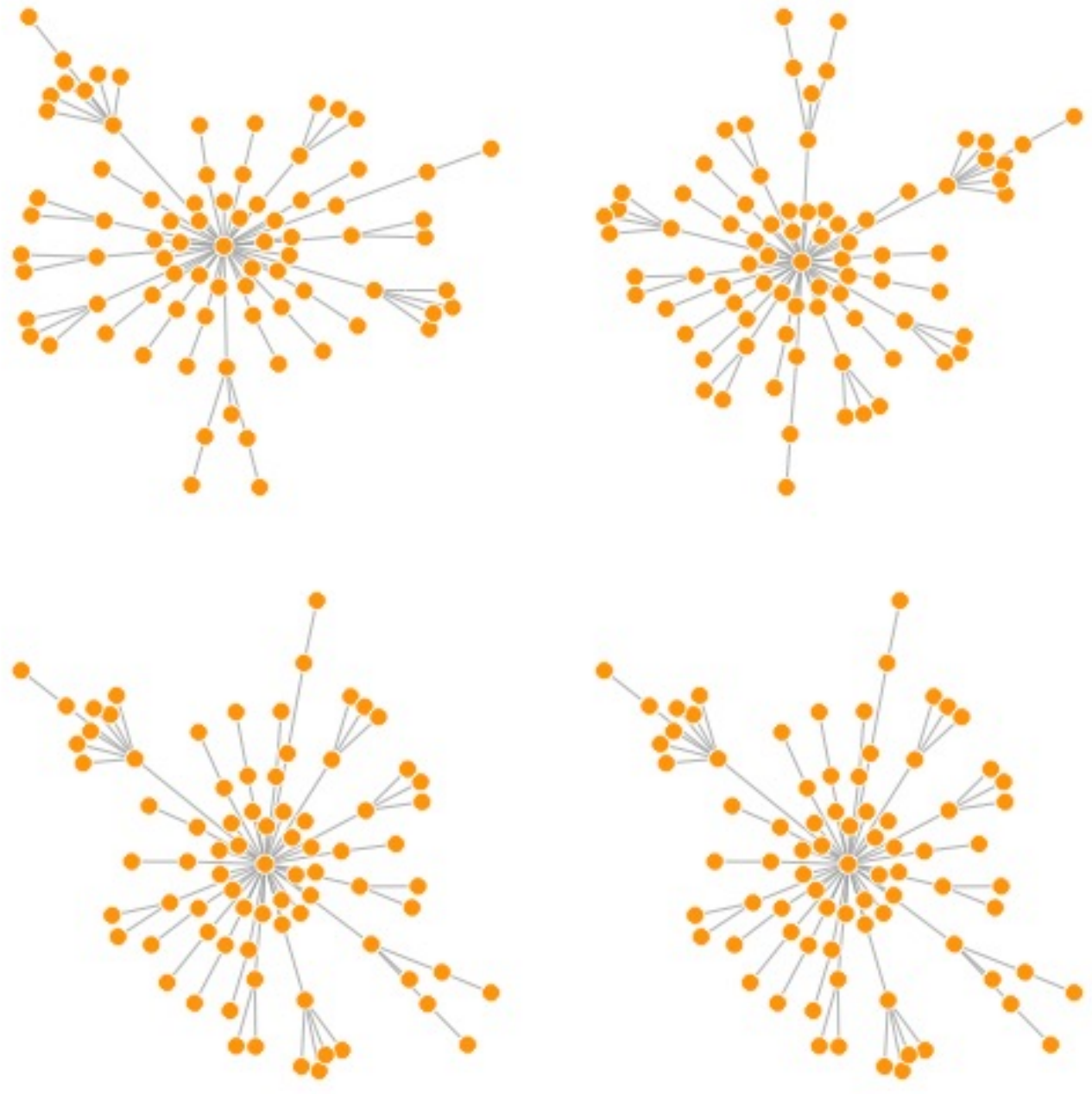
Layouts



Layouts

```
par(mfrow=c(2,2), mar=c(1,1,1,1))
plot(net.bg, layout=layout_with_fr)
plot(net.bg, layout=layout_with_fr)
plot(net.bg, layout=l)
plot(net.bg, layout=l)
```

Layouts



Layouts

Another popular force-directed algorithm that produces nice results for connected graphs is Kamada Kawai

```
l <- layout_with_kk(net.bg)
plot(net.bg, layout=l)
```

Layouts

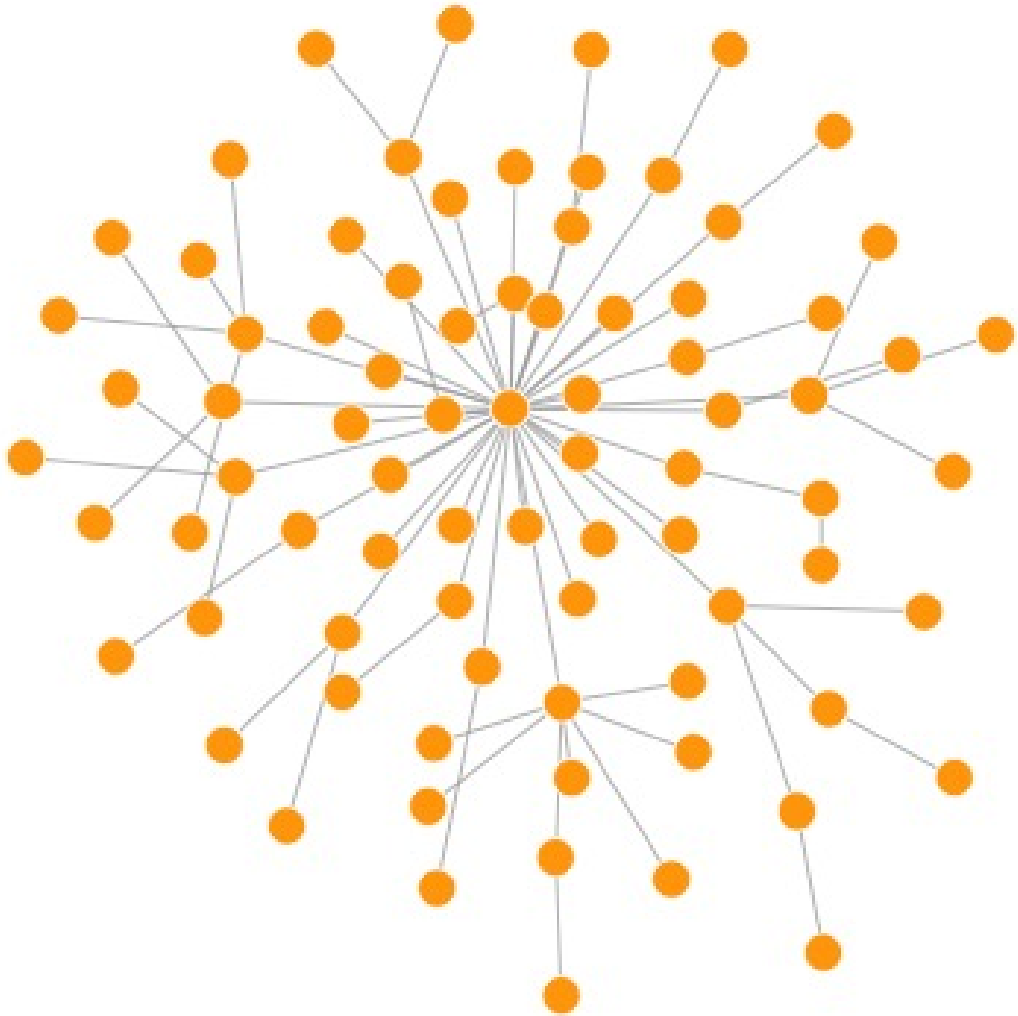


Layouts

- The LGL algorithm is for large connected graphs

```
1 plot(net.bg, layout=layout_with_lgl)
```

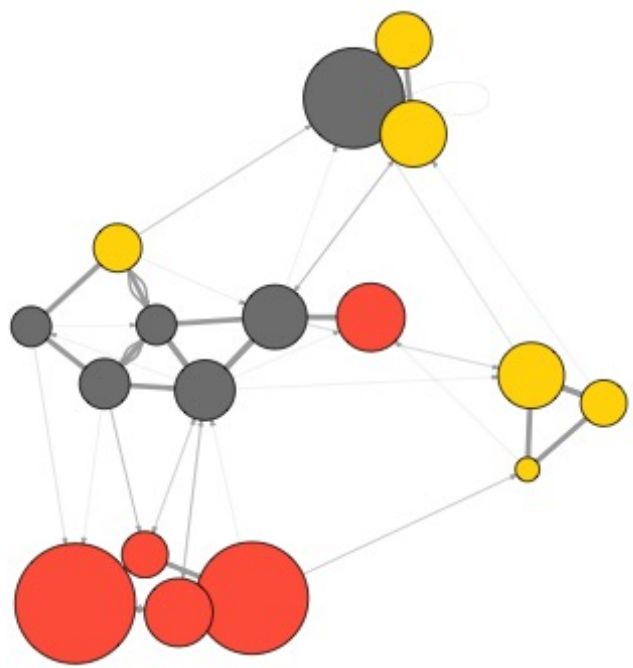
Layouts



Improving Network Analysis

- This network plot is still not too helpful
- We can identify the type and size of nodes, but cannot see much about the structure

```
1 plot(net)
```



Improving Network Analysis

```
mean(links$weight)
```

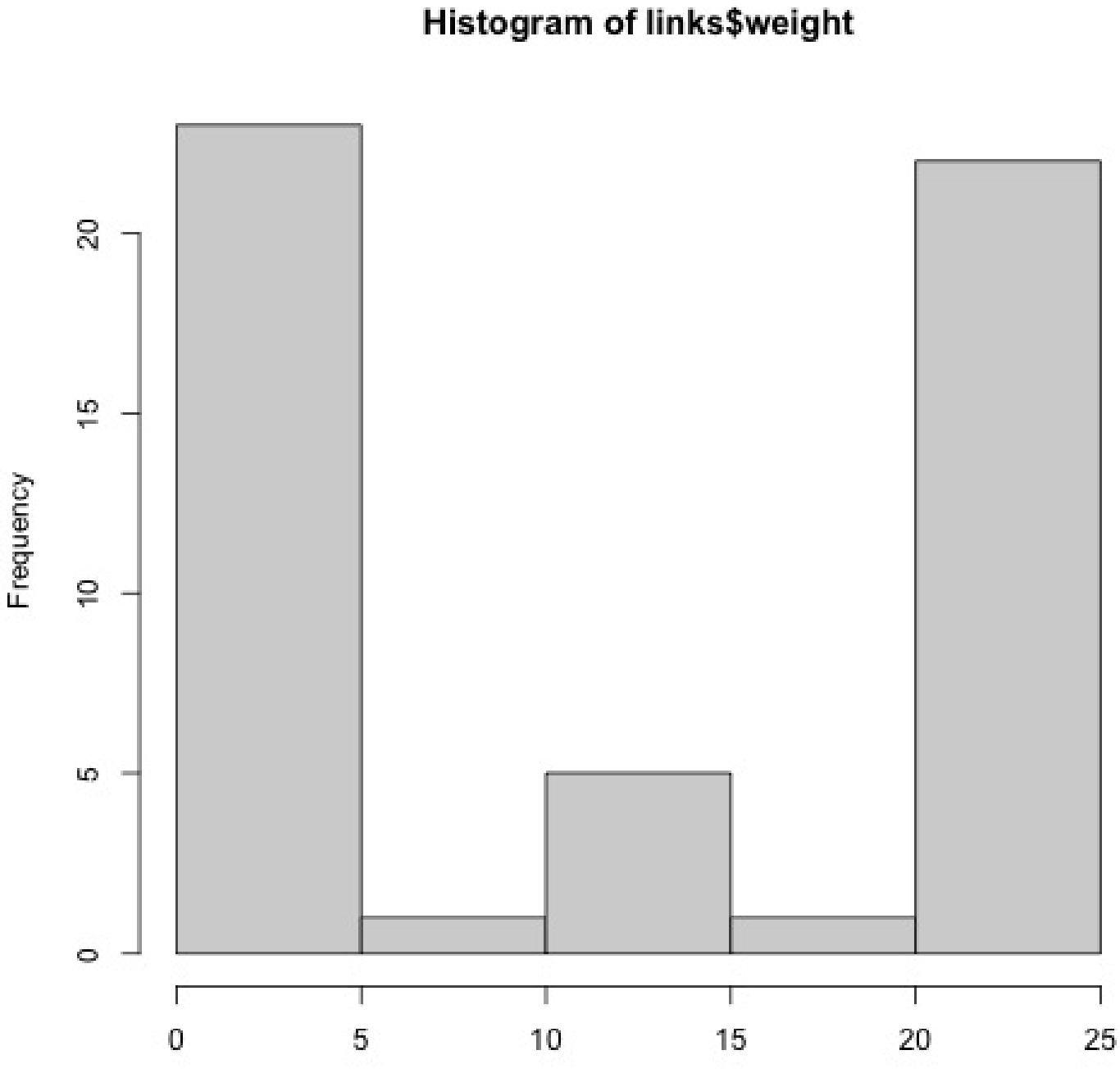
```
[1] 11.69231
```

```
sd(links$weight)
```

```
[1] 9.287433
```

```
hist(links$weight)
```

Improving Network Analysis

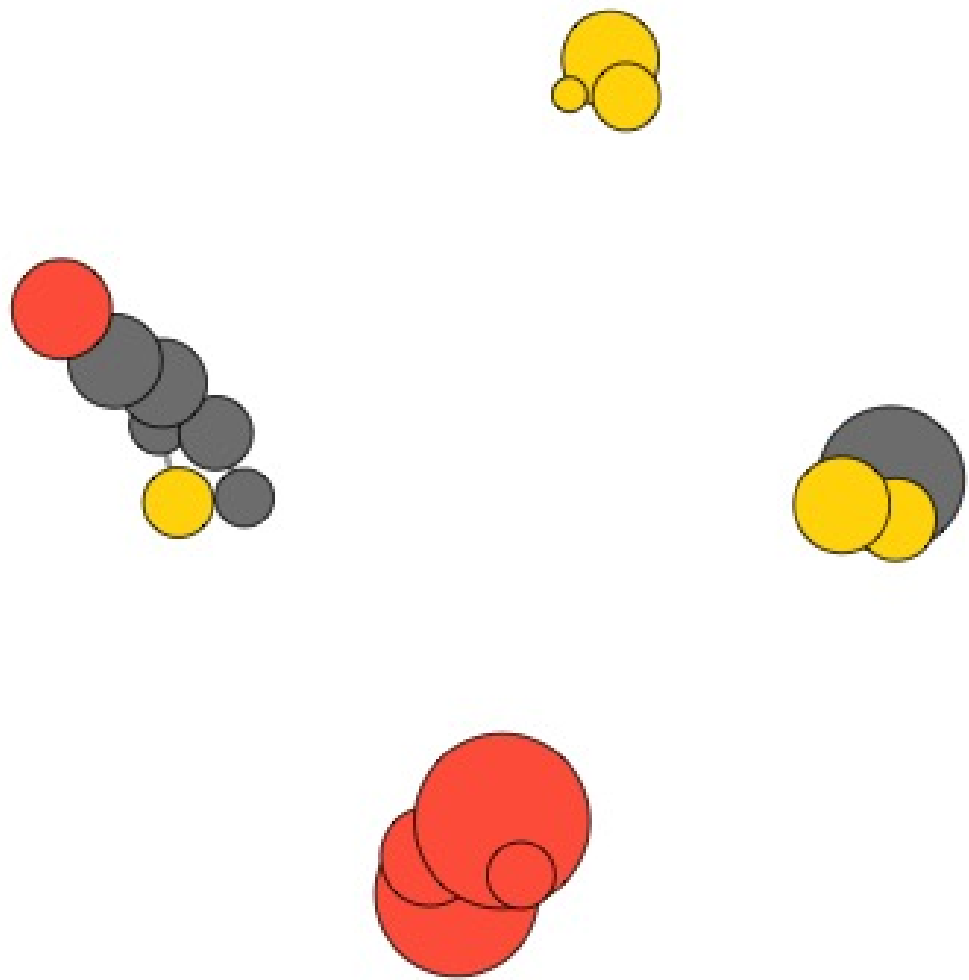


Improving Network Analysis

- There are more sophisticated ways to extract the key edges, but for the purposes of this exercise we keep the edges whose weight is above the average
- We can delete edges using delete_edges(net, edges)

```
cut.off <- mean(links$weight)
net.sp <- delete_edges(net, E(net)[weight<cut.off])
plot(net.sp)
```

Improving Network Analysis

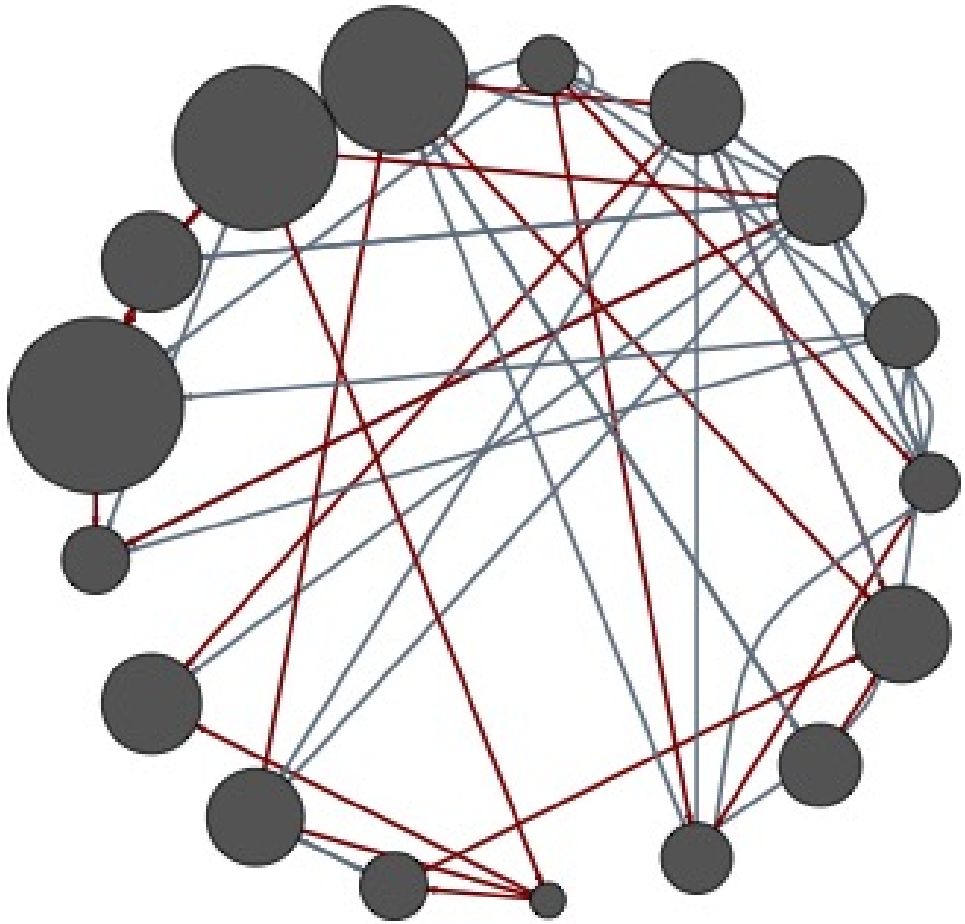


Improving Network Analysis

Or we can color the link types (hyperlik & mention):

```
E(net)$width <- 2
plot(net,
  edge.color=c("dark red",
               "slategrey")[(E(net)$type=="hyperlink")+1],
  vertex.color="gray40", layout=layout_in_circle)
```

Improving Network Analysis



Improving Network Analysis

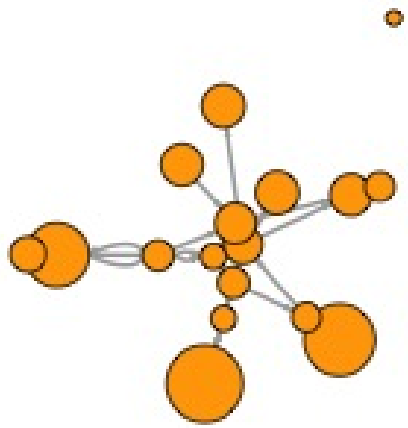
Another way to delete edges:

```
net.m <- net - E(net)[E(net)$type=="hyperlink"]
net.h <- net - E(net)[E(net)$type=="mention"]
par(mfrow=c(1,2))

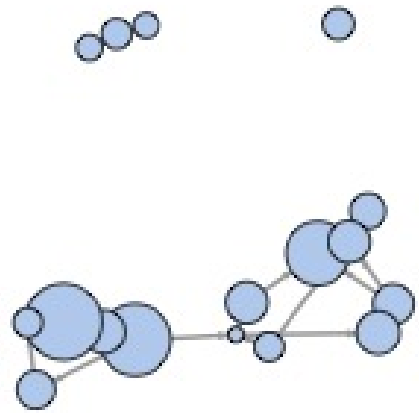
plot(net.h, vertex.color="orange",
     main="Link: Hyperlink")
plot(net.m, vertex.color="lightsteelblue2",
     main="Link: Mention")
```

Improving Network Analysis

Link: Hyperlink



Link: Mention



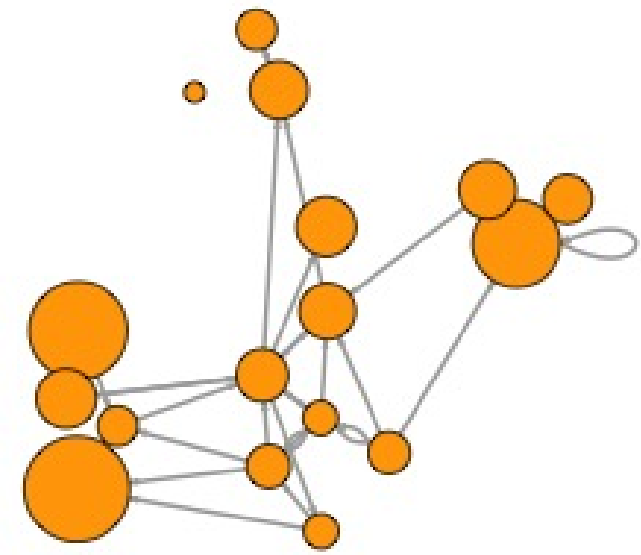
Improving Network Analysis

Make sure the nodes stay in place in both plots:

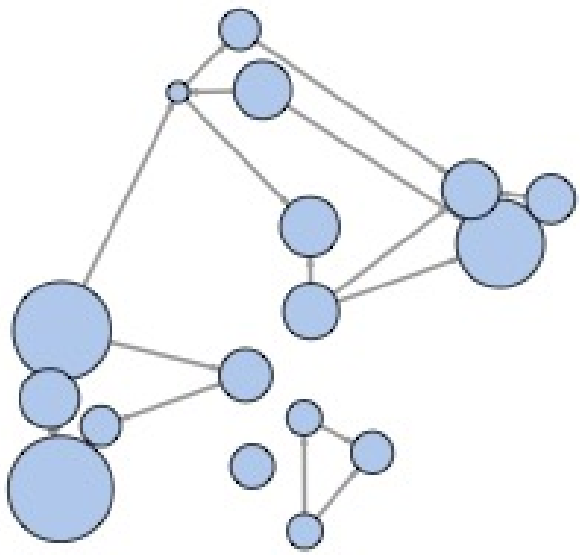
```
par(mfrow=c(1,2),mar=c(1,1,4,1))
l <- layout_with_fr(net)
plot(net.h, vertex.color="orange", layout=l,
     main="Tie: Hyperlink")
plot(net.m, vertex.color="lightsteelblue2",
     layout=l, main="Tie: Mention")
```

Improving Network Analysis

Tie: Hyperlink



Tie: Mention



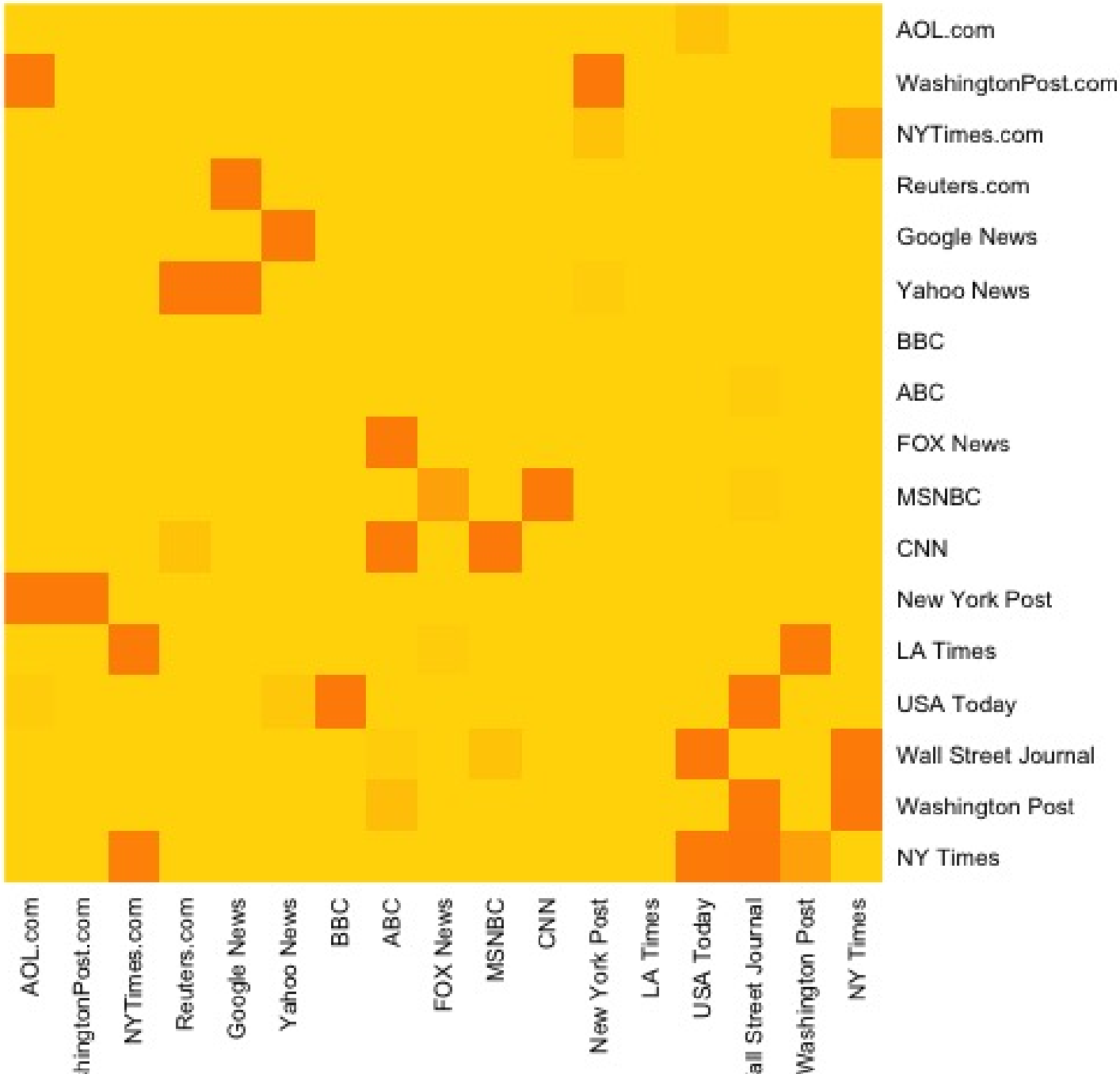
Heatmaps

A quick reminder that there are other ways to represent a network:

```
# Heatmap of the network matrix:
netm <- get.adjacency(net, attr="weight", sparse=F)
colnames(netm) <- V(net)$media
rownames(netm) <- V(net)$media

palf <- colorRampPalette(c("gold", "dark orange"))
heatmap(netm[,17:1], Rowv = NA, Colv = NA, col = palf(20),
        scale="none", margins=c(10,10) )
```

Improving Network Analysis



Notes

This presentation is partially based on:

- NetSci X Tutorial "Network Analysis and Visualization with R and igraph" by Katherine Ognyanova, www.kateto.net, NetSciX 2016 School of Code Workshop, Wroclaw, Poland
- Chapter 6 of Arlnold and Tilton, *Humanities Data in R. Exploring Networks, Geospatial Data, Images, and Text.* © 2015 Springer

All code in the presentation is licensed under the Apache License, Version 2.0 (the "License"); you may not use this file except in compliance with the License. You may obtain a copy of the License at http://www.apache.org/licenses/LICENSE-2.0 Unless required by applicable law or agreed to in writing, software distributed under the License is distributed on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the License for the specific language governing permissions and limitations under the License.